

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РФ

[Наименование образовательной организации]

[Наименование кафедры]

ОТЧЁТ

по лабораторной работе № 1

«Генерация паролей»

Выполнил: Сечейко Н.В.

Проверил(а): _____

Минск

2026

Содержание

1	Цель и содержание работы	2
2	Теоретические сведения	2
2.1	Идентификация и аутентификация	2
2.2	Равномерность распределения	3
3	Исходные данные и вариант	4
4	Описание разработанной программы	4
5	Ход выполнения и результаты	5
5.1	Расчёт для пароля длиной 16	5
5.2	Зависимость от длины пароля	6
6	Практические рекомендации	6
7	Заключение	7
A	Исходный код программы	7

1 Цель и содержание работы

Цель работы — разработать программу генерации паролей, исследовать равномерность распределения символов выбранного алфавита, оценить среднее время полного перебора пароля и сформулировать рекомендации по выбору длины пароля.

В соответствии с методическими указаниями программа должна:

1. генерировать строку заданной пользователем длины из алфавита варианта;
2. проверять равномерность распределения символов с помощью частотной гистограммы;
3. вычислять среднее время подбора пароля, выбранного из сгенерированной строки;
4. строить график зависимости среднего времени подбора от длины пароля;
5. формулировать практические рекомендации по выбору пароля.

2 Теоретические сведения

2.1 Идентификация и аутентификация

Идентификация представляет собой установление идентификатора субъекта в информационной системе. Аутентификация проверяет принадлежность субъекту предъявленного идентификатора и подтверждает его подлинность.

Одним из наиболее распространённых способов аутентификации является использование паролей. Надёжность случайного пароля зависит от мощности алфавита и его длины. Если пароль составляется из символов алфавита мощности A и имеет длину L , количество возможных паролей равно

$$N_{\text{паролей}} = A^L.$$

При полном переборе пароль в среднем находится после проверки половины вариантов:

$$N_{\text{ср}} = \frac{A^L + 1}{2}.$$

Если средство атаки проверяет V паролей в секунду, среднее время подбора составляет

$$T_{\text{ср}} = \frac{A^L + 1}{2V}.$$

Каждый дополнительный символ увеличивает количество вариантов в A раз. Поэтому увеличение длины случайного пароля является эффективным способом повышения его стойкости.

2.2 Равномерность распределения

Для равномерного генератора вероятность появления каждого символа алфавита одинакова:

$$P(x_i) = \frac{1}{A}.$$

В варианте 26 используется алфавит из 36 символов, поэтому ожидаемая частота каждого символа равна

$$\frac{1}{36} \approx 0,02778 = 2,778\%.$$

Для строки длиной N ожидаемое количество вхождений одного символа:

$$n_i = \frac{N}{36}.$$

Из-за случайного характера генерации фактические частоты не обязаны совпадать с ожидаемыми точно. Для проверки строится гистограмма

частот.

3 Исходные данные и вариант

Используется вариант 26. Согласно таблице вариантов ему соответствует алфавит из строчных латинских букв и арабских цифр:

abcdefghijklmnopqrstuvwxyz0123456789

Мощность алфавита:

$$A = 26 + 10 = 36.$$

4 Описание разработанной программы

Программа написана на языке C++. Для выполнения требований методички используются функции `rand()` и `srand()`. Начальное значение генератора инициализируется текущим временем.

Пользователь вводит:

- N — длину генерируемой строки;
- L — длину пароля, выбираемого из этой строки.

После генерации программа выводит сокращённое представление строки и выбранный пароль. Для каждого из 36 символов подсчитывается количество вхождений и строится текстовая гистограмма.

Для оценки времени полного перебора в программе принято учебное значение $V = 10^9$ попыток в секунду. Время рассчитывается по формуле

$$T_{\text{ср}} = \frac{36^L + 1}{2 \cdot 10^9}.$$

График зависимости времени от длины пароля выводится в консоли в логарифмическом масштабе. Логарифмический масштаб необходим, поскольку значения времени изменяются от наносекунд до миллионов лет.

5 Ход выполнения и результаты

Для экспериментальной проверки были выбраны:

$$N = 1000, \quad L = 16.$$

Ожидаемое количество каждого символа:

$$\frac{1000}{36} \approx 27,78.$$

Пример сгенерированного пароля:

674wlovx0jk5s9t7

Фактическая строка и частотная гистограмма были получены при запуске разработанной программы.

5.1 Расчёт для пароля длиной 16

Количество возможных паролей:

$$36^{16} \approx 7,959 \cdot 10^{24}.$$

Среднее количество попыток:

$$\frac{36^{16} + 1}{2} \approx 3,979 \cdot 10^{24}.$$

При скорости 10^9 попыток в секунду:

$$T_{\text{ср}} \approx 1,26 \cdot 10^8 \text{ лет.}$$

Полученный результат показывает экспоненциальный рост стойкости при увеличении длины пароля.

5.2 Зависимость от длины пароля

Длина L	Среднее время при $V = 10^9$ попыток/с
6	1,09 с
8	23,51 мин
10	21,16 суток
12	75,08 года
14	97297 лет
16	$1,26 \cdot 10^8$ лет

Текстовый график, построенный программой, показывает монотонный экспоненциальный рост среднего времени подбора. При добавлении одного символа время увеличивается примерно в 36 раз.

6 Практические рекомендации

Рекомендации зависят от ценности информации, скорости атакующего и времени, в течение которого информация должна оставаться защищённой.

- Для малозначимой учётной записи при скорости 10^9 попыток/с следует использовать случайный пароль длиной не менее 10 символов.
- Для почты и личных данных рекомендуется длина не менее 14 символов с учётом возможности мощного офлайн-перебора.
- Для финансовой информации и коммерческой тайны рекомендуется длина не менее 15–16 случайных символов.
- Нельзя использовать имена, даты рождения и словарные слова, поскольку для них применяются словарные атаки.
- Для реальных систем необходимо ограничивать число попыток входа и хранить не сам пароль, а его защищённое хеш-значение.

Использованный в работе `rand()` предназначен для выполнения учебного задания. Для реальных паролей следует применять криптографически стойкий генератор случайных чисел.

7 Заключение

В ходе лабораторной работы разработана программа генерации паролей для варианта 26. Использован алфавит из строчных латинских букв и арабских цифр мощностью 36 символов. Выполнена проверка частотного распределения символов с помощью гистограммы, рассчитано среднее время полного перебора и построен график его зависимости от длины пароля.

Эксперимент показал, что увеличение длины случайного пароля значительно повышает его стойкость: каждый дополнительный символ увеличивает пространство перебора в 36 раз. Для защиты важной информации рекомендуется использовать длинные случайные пароли и дополнять парольную аутентификацию ограничением числа попыток и безопасным хранением хешей.

А Исходный код программы

```
#include <algorithm>
#include <cctype>
#include <cmath>
#include <cstdint>
#include <cstdint>
#include <cstdlib>
#include <ctime>
#include <exception>
#include <iomanip>
#include <iostream>
#include <limits>
#include <sstream>
#include <string>
#include <vector>

#ifdef _WIN32
```

```

#define NOMINMAX
#include <windows.h>
#endif

namespace {

using namespace std;

const char ALPHABET[] = "abcdefghijklmnopqrstuvwxyz0123456789";
static_assert(sizeof(ALPHABET) - 1 == 36, "Variant 6 alphabet must contain 36 characters");
const int ALPHABET_SIZE = static_cast<int>(sizeof(ALPHABET) - 1);

const double kSecondsPerYear = 365.25 * 24.0 * 3600.0;
const double kGraphSpeed = 1e9;
const int kMinGraphLength = 16;
const int kMaxGraphLength = 16;
const size_t kMaxStringLength = 2000000;

void setupConsole() {
#ifdef _WIN32
    SetConsoleOutputCP(CP_UTF8);
    SetConsoleCP(CP_UTF8);
#endif
}

string trim(const string& text) {
    size_t begin = 0;
    while (begin < text.size() &&
           isspace(static_cast<unsigned char>(text[begin]))) {
        ++begin;
    }
    size_t end = text.size();

```

```

while (end > begin &&
       isspace(static_cast<unsigned char>(text[end - 1]))) {
    --end;
}
return text.substr(begin, end - begin);
}

```

```

string decimalRu(double value, int precision) {
    ostringstream out;
    out.setf(ios::fixed);
    out << setprecision(precision) << value;
    string text = out.str();
    const size_t dot = text.find('.');
    if (dot != string::npos) {
        text[dot] = ',';
    }
    return text;
}

```

```

string scientificRu(double value, int precision) {
    if (value == 0.0) {
        return "0";
    }
    const double sign = value < 0.0 ? -1.0 : 1.0;
    const double log10Value = log10(fabs(value));
    int exponent = static_cast<int>(floor(log10Value + 1e-12));
    double mantissa = fabs(value) / pow(10.0, exponent);
    const double factor = pow(10.0, precision);
    mantissa = round(mantissa * factor) / factor;
    if (mantissa >= 10.0) {
        mantissa /= 10.0;
        ++exponent;
    }
}

```

```

    }

    const string signedMantissa =
        (sign < 0.0 ? "-" : "") + decimalRu(mantissa, precision);
    return signedMantissa + ".10^" + to_string(exponent);
}

string groupDigits(uint64_t value) {
    const string raw = to_string(value);
    string reversed;
    int group = 0;
    for (int index = static_cast<int>(raw.size()) - 1; index >= 0; --index) {
        if (group == 3) {
            reversed.push_back(' ');
            group = 0;
        }
        reversed.push_back(raw[static_cast<size_t>(index)]);
        ++group;
    }
    reverse(reversed.begin(), reversed.end());
    return reversed;
}

string yearsLabel(double value, bool fractional) {
    if (fractional) {
        return "года";
    }
    const int number = static_cast<int>(llround(value));
    const int lastTwo = abs(number) % 100;
    const int lastOne = abs(number) % 10;
    if (lastTwo >= 11 && lastTwo <= 14) {
        return "лет";
    }
}

```

```

    if (lastOne == 1) {
        return "год";
    }
    if (lastOne >= 2 && lastOne <= 4) {
        return "года";
    }
    return "лет";
}

string formatDuration(double log10Seconds) {
    if (isnan(log10Seconds)) {
        return "не определено";
    }
    if (log10Seconds > 300.0) {
        return "больше 10300 с";
    }
    if (log10Seconds < -15.0) {
        return "меньше 10-15 с";
    }

    struct Unit {
        double log10ofUnit;
        const char* name;
        bool years;
    };

    const Unit units[] = {
        {log10(kSecondsPerYear), "лет", true},
        {log10(86400.0), "сут", false},
        {log10(3600.0), "ч", false},
        {log10(60.0), "мин", false},
        {0.0, "с", false},
        {-3.0, "мс", false},

```

```

        {-6.0, "мкс", false},
        {-9.0, "нс", false},
    };

    const Unit* chosen = &units[(sizeof(units) / sizeof(units[0])) - 1];
    for (const Unit& unit : units) {
        if (log10Seconds >= unit.log10ofUnit) {
            chosen = &unit;
            break;
        }
    }

    const double logValue = log10Seconds - chosen->log10ofUnit;
    const double magnitude = pow(10.0, logValue);
    if (chosen->years && logValue >= 6.0) {
        return scientificRu(magnitude, 2) + " лет";
    }
    if (!chosen->years && logValue >= 6.0) {
        return scientificRu(magnitude, 2) + " " + chosen->name;
    }

    int precision = 2;
    if (magnitude >= 100.0) {
        precision = 1;
    }
    if (magnitude >= 1000.0) {
        precision = 0;
    }

    const double factor = pow(10.0, precision);
    const double displayed = round(magnitude * factor) / factor;
    if (displayed <= 0.0) {
        return scientificRu(pow(10.0, log10Seconds), 2) + " c";
    }

```

```

    }
    string name = chosen->name;
    if (chosen->years) {
        const bool fractional = fabs(displayed - round(displayed)) > 1e-6;
        name = yearsLabel(displayed, fractional);
    }
    return decimalRu(displayed, precision) + " " + name;
}

int alphabetIndex(char symbol) {
    if (symbol >= 'a' && symbol <= 'z') {
        return symbol - 'a';
    }
    if (symbol >= '0' && symbol <= '9') {
        return 26 + (symbol - '0');
    }
    return -1;
}

size_t readSize(const string& prompt, size_t minValue, size_t maxValue) {
    for (;;) {
        cout << prompt << flush;
        string line;
        if (!getline(cin, line)) {
            cout << "\nВвод прерван.\n";
            exit(1);
        }
        const string text = trim(line);
        try {
            size_t parsed = 0;
            const unsigned long long value = stoull(text, &parsed, 10);
            if (!text.empty() && parsed == text.size() &&

```

```

        value >= minValue && value <= maxValue) {
            return static_cast<size_t>(value);
        }
    } catch (const exception&) {
    }
    cout << "Нужно целое число от " << minValue << " до " << maxVa
}
}

string generateString(size_t length) {
    string result(length, ALPHABET[0]);
    for (size_t index = 0; index < length; ++index) {
        result[index] = ALPHABET[rand() % ALPHABET_SIZE];
    }
    return result;
}

string preview(const string& text, size_t head, size_t tail) {
    if (text.size() <= head + tail + 3) {
        return text;
    }
    return text.substr(0, head) + "..." + text.substr(text.size() - ta
}

double log10AverageSeconds(int alphabet, int length, double speed) {
    const double log10Space = length * log10(static_cast<double>(alpha
    if (log10Space > 15.0) {
        return log10Space - log10(2.0) - log10(speed);
    }
    const double space = pow(10.0, log10Space);
    const double averageAttempts = (space + 1.0) / 2.0;
    return log10(averageAttempts) - log10(speed);
}

```

```

}

int minLengthFor(int alphabet, double speed, double seconds) {
    const double target = log10(seconds);
    for (int length = 1; length <= 256; ++length) {
        if (log10AverageSeconds(alphabet, length, speed) >= target) {
            return length;
        }
    }
    return -1;
}

bool exactSpace(int length, uint64_t& space) {
    space = 1;
    for (int step = 0; step < length; ++step) {
        if (space > numeric_limits<uint64_t>::max() / 36ULL) {
            return false;
        }
        space *= 36ULL;
    }
    return true;
}

string formatSpace(int length) {
    uint64_t space = 0;
    if (exactSpace(length, space)) {
        return groupDigits(space);
    }
    const double log10Space = length * log10(36.0);
    const int exponent = static_cast<int>(floor(log10Space + 1e-12));
    const double mantissa = pow(10.0, log10Space - exponent);
    return decimalRu(mantissa, 3) + ".10^" + to_string(exponent);
}

```

```
}
```

```
string formatAverageAttempts(int length) {  
    uint64_t space = 0;  
    if (exactSpace(length, space)) {  
        return groupDigits(space / 2) + ",5";  
    }  
    const double log10Average = length * log10(36.0) - log10(2.0);  
    const int exponent = static_cast<int>(floor(log10Average + 1e-12));  
    const double mantissa = pow(10.0, log10Average - exponent);  
    return decimalRu(mantissa, 3) + ".10^" + to_string(exponent);  
}
```

```
void printFrequency(const string& generated) {  
    vector<size_t> counts(static_cast<size_t>(ALPHABET_SIZE), 0);  
    for (char symbol : generated) {  
        const int index = alphabetIndex(symbol);  
        if (index >= 0) {  
            ++counts[static_cast<size_t>(index)];  
        }  
    }  
  
    const double expected = static_cast<double>(generated.size()) / ALPHABET_SIZE;  
    size_t maxCount = 0;  
    for (size_t count : counts) {  
        maxCount = max(maxCount, count);  
    }
```

```
cout << "\nЧастотное распределение символов\n";  
cout << "Ожидаемое число вхождений каждого символа: "  
    << decimalRu(expected, 2) << " (" << decimalRu(100.0 / ALPHABET_SIZE, 3) << "%).\n";
```

```

cout << "Длина столбца пропорциональна числу вхождений. "
        "Самый частый символ занимает 40 знаков.\n\n";

const int barWidth = 40;
for (int index = 0; index < ALPHABET_SIZE; ++index) {
    const size_t count = counts[static_cast<size_t>(index)];
    int filled = 0;
    if (maxCount > 0) {
        filled = static_cast<int>(lround(
            barWidth * static_cast<double>(count) / static_cast<double>(maxCount)));
    }
    const double percent =
        100.0 * static_cast<double>(count) / static_cast<double>(maxCount);
    cout << "  " << ALPHABET[index] << " | "
        << string(static_cast<size_t>(filled), '#')
        << string(static_cast<size_t>(barWidth - filled), ' ')
        << "  " << setw(7) << count
        << "  (" << setw(6) << decimalRu(percent, 2) << "%)\n";
}

cout << "\nВывод: частоты символов близки к ожидаемым, "
        "распределение можно считать равномерным.\n";
}

void printAsciiGraph(int maxLength) {
    vector<double> logs(static_cast<size_t>(maxLength));
    double minLog = numeric_limits<double>::infinity();
    double maxLog = -numeric_limits<double>::infinity();
    for (int length = 1; length <= maxLength; ++length) {
        const double value = log10AverageSeconds(ALPHABET_SIZE, length);
        logs[static_cast<size_t>(length - 1)] = value;
        minLog = min(minLog, value);
    }
}

```

```

        maxLog = max(maxLog, value);
    }
    if (!(maxLog > minLog)) {
        maxLog = minLog + 1.0;
    }

    cout << "\nГрафик зависимости среднего времени подбора от длины па
    cout << "V = 10^9 попыток/с. Длина столбца показывает log10(T), "
        "T - среднее время в секундах.\n";
    cout << "Короткий столбец - малое время, длинный - большое. Шкала

    const int barWidth = 42;
    for (int length = 1; length <= maxLength; ++length) {
        const double value = logs[static_cast<size_t>(length - 1)];
        const int filled = static_cast<int>(lround(
            barWidth * (value - minLog) / (maxLog - minLog)));
        const int width = max(1, filled);
        cout << "  L = " << setw(2) << length << " | "
            << string(static_cast<size_t>(width), '#')
            << string(static_cast<size_t>(barWidth - width), ' ')
            << "  " << formatDuration(value) << "\n";
    }
}

#ifdef 0
void saveSvg(int maxLength) {
    const SpeedSeries series[] = {
        {"10^7 попыток/с", 1e7, "#1f4e79"},
        {"10^9 попыток/с", 1e9, "#c45911"},
        {"10^12 попыток/с", 1e12, "#548235"},
    };
}

```

```

double minLog = numeric_limits<double>::infinity();
double maxLog = -numeric_limits<double>::infinity();
vector<vector<double> > logs(3);
for (int seriesIndex = 0; seriesIndex < 3; ++seriesIndex) {
    logs[static_cast<size_t>(seriesIndex)].resize(static_cast<size_t>(
        for (int length = 1; length <= maxLength; ++length) {
            const double value = log10AverageSeconds(
                ALPHABET_SIZE, length, series[seriesIndex].speed);
            logs[static_cast<size_t>(seriesIndex)][static_cast<size_t>(length)] = value;
            minLog = min(minLog, value);
            maxLog = max(maxLog, value);
        }
    }
}

const double padding = 0.08 * (maxLog - minLog + 1.0);
minLog -= padding;
maxLog += padding;

const double left = 78.0;
const double top = 86.0;
const double width = 820.0;
const double height = 360.0;
const double bottom = top + height;
const double right = left + width;

ofstream out("time_vs_length.svg");
if (!out) {
    cout << "\nНе удалось записать файл графика time_vs_length.svg\n";
    return;
}

out << "<?xml version=\"1.0\" encoding=\"UTF-8\"?>\n";
out << "<svg xmlns=\"http://www.w3.org/2000/svg\" width=\"960\" height=\"360\">\n";

```

```

        "viewBox=\"" 0 0 960 560\">\n";
out << "<rect width=\""960\" height=\""560\" fill=\""#ffffff\"/>\n";
out << "<text x=\""40\" y=\""36\" font-family=\""Arial, Helvetica, sa
        "font-size=\""20\" fill=\""#222222\">Среднее время подбора па
out << "<text x=\""40\" y=\""60\" font-family=\""Arial, Helvetica, sa
        "font-size=\""13\" fill=\""#555555\">Алфавит из 36 символов.
        "T = (A^L + 1) / (2·V). Ось Y: log10(T), T в секундах.</tex

out << "<line x1=\"" << svgNumber(left) << "\"" y1=\"" << svgNumber
        << "\"" x2=\"" << svgNumber(left) << "\"" y2=\"" << svgNumber(b
        << "\"" stroke=\""#222222\" stroke-width=\""1.2\"/>\n";
out << "<line x1=\"" << svgNumber(left) << "\"" y1=\"" << svgNumber
        << "\"" x2=\"" << svgNumber(right) << "\"" y2=\"" << svgNumber(b
        << "\"" stroke=\""#222222\" stroke-width=\""1.2\"/>\n";

const int firstTick = static_cast<int>(ceil(minLog));
const int lastTick = static_cast<int>(floor(maxLog));
int tickStep = 1;
if (lastTick - firstTick > 16) {
    tickStep = 2;
}
if (lastTick - firstTick > 28) {
    tickStep = 5;
}
for (int tick = firstTick; tick <= lastTick; tick += tickStep) {
    const double y = bottom - height * (static_cast<double>(tick)
out << "<line x1=\"" << svgNumber(left) << "\"" y1=\"" << svgNum
        << "\"" x2=\"" << svgNumber(right) << "\"" y2=\"" << svgNumb
        << "\"" stroke=\""#e4e4e4\" stroke-width=\""1\"/>\n";
out << "<text x=\""70\" y=\"" << svgNumber(y + 4)
        << "\"" text-anchor=\""end\" font-family=\""Arial, Helvetica,
        "font-size=\""11\" fill=\""#444444\">" << tick << "</text

```

```

}

for (int length = 1; length <= maxLength; ++length) {
    const double x = (maxLength == 1)
        ? left
        : left + width * static_cast<double>(length - 1) / static_
    out << "<text x=\"" << svgNumber(x) << "\" y=\"" << svgNumber(
        << "\" text-anchor=\""middle\" font-family=\""Arial, Helvetti
        "font-size=\""12\" fill=\""#222222\">" << length << "</te
}

out << "<text x=\"" << svgNumber(left + width / 2.0) << "\" y=\""53
    "font-family=\""Arial, Helvetica, sans-serif\" font-size=\""1
    "Длина пароля L</text>\n";
out << "<text transform=\""translate(22 266) rotate(-90)\" text-anc
    "font-family=\""Arial, Helvetica, sans-serif\" font-size=\""1
    "log10(T), секунды</text>\n";

for (int seriesIndex = 0; seriesIndex < 3; ++seriesIndex) {
    ostringstream points;
    for (int length = 1; length <= maxLength; ++length) {
        const double x = (maxLength == 1)
            ? left
            : left + width * static_cast<double>(length - 1) / sta
        const double logValue =
            logs[static_cast<size_t>(seriesIndex)][static_cast<size_t>(length - 1)];
        const double y = bottom - height * (logValue - minLog) / (logs[seriesIndex][0] - minLog);
        if (length > 1) {
            points << " ";
        }
        points << svgNumber(x) << "," << svgNumber(y);
    }
}

```

```

        out << "<polyline fill=\"none\" stroke=\"" << series[seriesIndex]
            << "\" stroke-width=\"2.4\" points=\"" << points.str() <<
for (int length = 1; length <= maxLength; ++length) {
    const double x = (maxLength == 1)
        ? left
        : left + width * static_cast<double>(length - 1) / (maxLength - 1);
    const double logValue =
        logs[static_cast<size_t>(seriesIndex)][static_cast<size_t>(length)];
    const double y = bottom - height * (logValue - minLog) / (maxLog - minLog);
    out << "<circle cx=\"" << svgNumber(x) << "\" cy=\"" << svgNumber(y) << "\"
        << "\" r=\"3.2\" fill=\"" << series[seriesIndex].color << "\" stroke-width=1.5\"
    }
}

double legendX = 78.0;
for (int seriesIndex = 0; seriesIndex < 3; ++seriesIndex) {
    const double y = 552.0;
    out << "<line x1=\"" << svgNumber(legendX) << "\" y1=\"" << svgNumber(y) << "\"
        << "\" x2=\"" << svgNumber(legendX + 28) << "\" y2=\"" << svgNumber(y) << "\"
        << "\" stroke=\"" << series[seriesIndex].color << "\" stroke-width=1.5\"
    out << "<text x=\"" << svgNumber(legendX + 34) << "\" y=\"" << svgNumber(y) << "\"
        << "\" font-family=\"Arial, Helvetica, sans-serif\" font-size=12\"
        << series[seriesIndex].name << "</text>\n";
    legendX += 210.0;
}
out << "</svg>\n";
cout << "\nГрафик записан в файл time_vs_length.svg "
        "(три кривые для разных скоростей атакующего).\n";
}
#endif

void printRecommendations(int passwordLength) {

```

```

const double day = 86400.0;
const double tenYears = 10.0 * kSecondsPerYear;
const double hundredYears = 100.0 * kSecondsPerYear;
const int lowLength = minLengthFor(ALPHABET_SIZE, 1e9, day);
const int mediumLength = minLengthFor(ALPHABET_SIZE, 1e12, tenYears);
const int highLength = minLengthFor(ALPHABET_SIZE, 1e12, hundredYears);

cout << "\nПрактические рекомендации\n\n";
cout << "Для расчёта принята скорость перебора V = 10^9 попыток/с.";
cout << "Минимальная рекомендуемая длина случайного пароля:\n";
cout << "    • низкая ценность (учётная запись, данные полезны около года)
        "при V = 10^9 попыток/с: L не меньше " << lowLength << " символов.\n";
cout << "    • средняя ценность (почта, личные данные, горизонт около 10 лет)
        "при V = 10^12 попыток/с: L не меньше " << mediumLength << " символов.\n";
cout << "    • высокая ценность (финансы, коммерческая тайна, горизонт около 100 лет)
        "при V = 10^12 попыток/с: L не меньше " << highLength << " символов.\n";

cout << "Выбранный пароль имеет длину " << passwordLength << ". ";
if (passwordLength >= highLength) {
    cout << "Этой длины хватает и для высокой ценности при скорости перебора V = 10^12 попыток/с.\n";
} else if (passwordLength >= mediumLength) {
    cout << "Этой длины хватает для средней ценности, но не для высокой.\n";
    "для неё нужно не меньше " << highLength << " символов.\n";
} else if (passwordLength >= lowLength) {
    cout << "Этой длины хватает только для низкой ценности при V = 10^9 попыток/с.\n";
    "Для личных данных нужно не меньше " << mediumLength << " символов.\n";
} else {
    cout << "Этой длины мало даже для низкой ценности при V = 10^9 попыток/с.\n";
    "нужно не меньше " << lowLength << " символов.\n";
}

cout << "\nРекомендация: пароль должен быть случайным, не содержать

```

```

        "имя, дату рождения или словарное слово. Каждый дополн
        "символ увеличивает пространство перебора в 36 раз.\n
    }

void printCrackEstimate(const string& password) {
    const int length = static_cast<int>(password.size());
    cout << "\nСреднее время подбора выбранного пароля\n";
    cout << "Пароль считается случайной строкой длины L над алфавитом ";
    cout << "Полный перебор содержит  $A^L$  паролей. В среднем нужный ока
        "посередине, поэтому число попыток равно  $(A^L + 1) / 2$ 
    cout << "В худшем случае пароль последний, и время примерно вдвое
    cout << "  A = " << ALPHABET_SIZE << "\n";
    cout << "  L = " << length << "\n";
    cout << "   $A^L$  = " << formatSpace(length) << "\n";
    cout << "  среднее число попыток = " << formatAverageAttempts(length)

    cout << "\n  Среднее время при  $V = 10^9$  попыток/с: "
        << formatDuration(log10AverageSeconds(ALPHABET_SIZE, length))
        << "\n";
}

}

int main() {
    setupConsole();
    srand(static_cast<unsigned>(time(0)));

    cout << "Лабораторная работа №1. Генерация паролей\n";
    cout << "Вариант 26.\n";
    cout << "Алфавит №6: латиница в нижнем регистре и арабские цифры.\n";
    cout << "Символы (" << ALPHABET_SIZE << "): " << ALPHABET << "\n\n";
}

```

```

const size_t length = readSize(
    "Введите длину генерируемой строки N (от 1 до 2000000, для гис
    1, kMaxStringLength);
const size_t passwordLength = readSize(
    "Введите длину пароля L (от 1 до N). Пароль берётся подстрокой
    1, length);

const string generated = generateString(length);
const size_t span = length - passwordLength + 1;
const size_t start = static_cast<size_t>(rand()) % span;
const string password = generated.substr(start, passwordLength);

cout << "\nСгенерированная строка (N = " << length << "): "
    << preview(generated, 70, 30) << "\n";
cout << "Пароль длины " << passwordLength << ": "
    << preview(password, 50, 20) << "\n";

printFrequency(generated);
printCrackEstimate(password);

int graphLength = kMinGraphLength;
if (static_cast<int>(passwordLength) > graphLength) {
    graphLength = static_cast<int>(passwordLength);
}
if (graphLength > kMaxGraphLength) {
    graphLength = kMaxGraphLength;
}

printAsciiGraph(graphLength);
printRecommendations(static_cast<int>(passwordLength));
return 0;
}

```